

Stacking Handcrafted and Convolutional Features for Wafer Map Pattern Classification, Revisited: Reproduction, Meta-Training Pitfalls, and Improvements

Kartikeya Gupta

Indian Institute of Information Technology Sri City
Sri City, Andhra Pradesh, India
kartikeyagupta.r@gmail.com

Abstract—Kang and Kang (2021) proposed classifying wafer map defect patterns with a stacking ensemble that combines a feed-forward network on 59 handcrafted features with a VGG-style convolutional network, merged by a multi-response linear regression (MLR) meta-classifier. We present an independent reproduction of all six models of that study on the WM-811K dataset at four training-set sizes, and a set of controlled extensions. At full data, our reproduction reaches a macro-F1 of 0.8967 for Stacking-MLR against the reported 0.8949 ± 0.0121 , and matches the remaining baselines. The reproduction exposed one pitfall: when the meta-classifier is trained on the base classifiers’ predictions for their own training data, as in the public reference code, it learns to trust over-confident outputs and ignores the rarest class. Training it on a held-out split resolves this, and the same mechanism explains two departures from the original results under its protocol. Building on the corrected pipeline, we evaluate test-time augmentation, gradient-boosted trees on the handcrafted features, and CNN seed ensembles under a fixed inclusion rule with paired statistics. The best pipeline, a five-seed CNN ensemble with test-time augmentation and XGBoost combined by MLR, reaches a test macro-F1 of 0.9134 (accuracy 98.21%). With a confidence-based reject option it classifies 95.2% of wafers automatically at 99.72% accuracy. We also report negative results: once the CNN is an ensemble, the original handcrafted-feature network no longer adds information, and temperature scaling does not improve the calibration of the MLR stack.

Index Terms—wafer map, defect pattern classification, stacking ensemble, reproducibility, selective classification, WM-811K

I. INTRODUCTION

A wafer map records which dies on a semiconductor wafer passed or failed electrical tests. Failed dies often form spatial patterns—rings at the edge, a cluster at the center, a scratch—and each pattern points to a different process problem, so classifying the patterns automatically supports yield analysis [1]. Two families of methods dominate: classifiers on handcrafted features such as region densities, Radon-transform profiles and the geometry of the largest defect region [1]–[3], and convolutional neural networks (CNNs) that learn features from the map itself [4]–[8].

Kang and Kang [9] observed that the two families are complementary and combined them by stacking [10], [11]: an

MFE+FNN classifier on 59 manually extracted features and a VGG16-style CNN [12] are merged by a multi-response linear regression (MLR) meta-classifier that learns, per class, how much to trust each base classifier. On WM-811K [1], [13] they report that the stack outperforms both base classifiers and four further baselines.

Reproductions are a basic check on such claims [14]. This paper reports one, followed by extensions that test which parts of the pipeline matter. Our contributions are:

- 1) An independent reproduction of all six models of [9] on WM-811K, at the full training size and across the training-size sweep $N \in \{500, 5,000, 50,000\}$. At full data every model matches or exceeds the reported value (Section V).
- 2) The identification of a meta-training pitfall: fitting the meta-classifier on in-sample base-classifier outputs, as in the public reference implementation, leaves the stack blind to the rarest class. A held-out meta-training split fixes it and explains two departures under the original protocol (Section V-B).
- 3) A controlled evaluation of three extensions—test-time augmentation, an XGBoost learner, and CNN seed ensembles—with a pre-specified inclusion rule, paired t -tests and wafer-level bootstrap intervals, including negative results (Sections VI–IX).
- 4) A selective-prediction analysis showing how many wafers can be classified automatically at a target accuracy (Section VII).

II. RELATED WORK

Handcrafted features. Wu et al. [1] introduced WM-811K together with Radon-transform and geometry features and a support vector machine. Later work used tree ensembles on Radon features [2], voting ensembles [3] and discriminant analysis [15].

CNNs. CNNs trained on the maps themselves now dominate [4], [6], [7], with extensions for mixed-type patterns [5], rotation invariance [8], active learning [16], lightweight mod-

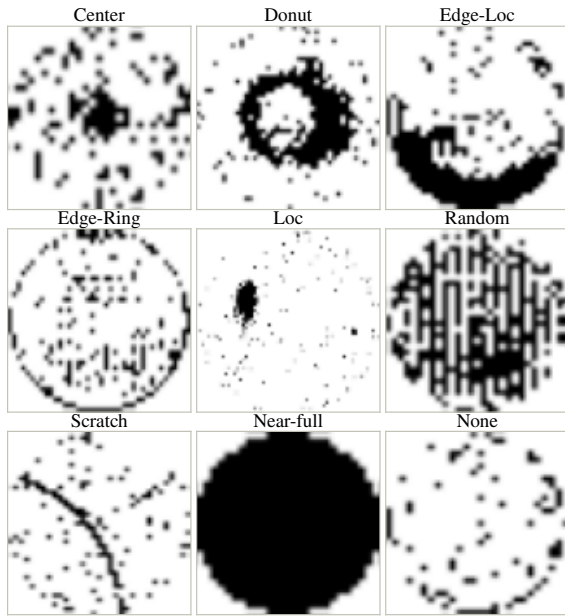


Fig. 1. One test wafer map per class (dark: failed die; 64×64 CNN input).

els with augmentation [17] and error-correcting output codes on CNN features [18].

Stacking. Stacked generalization [10] trains a meta-learner on base-learner outputs. Ting and Witten [11] showed that the meta-learner should be trained on outputs for data the base learners did not see (they used cross-validation) and that class-probability outputs with an MLR meta-learner work well; Džeroski and Ženko [19] compared stacking with selecting the best base model. Kang and Kang [9] applied MLR stacking to handcrafted and CNN wafer-map classifiers.

Ensembles, calibration and selective prediction. Averaging independently trained networks improves accuracy and uncertainty estimates [20]. Modern networks are often overconfident; temperature scaling [21] and the expected calibration error (ECE) [22] are the standard tools. Selective classification abstains on low-confidence inputs to reach a target risk [23].

III. DATA AND ORIGINAL METHOD

Data. WM-811K (LSWMD) [1], [13] holds 811,457 wafer maps, of which 172,950 are labeled with one of nine classes: Center, Donut, Edge-Loc, Edge-Ring, Loc, Random, Scratch, Near-full and None. Following [9], maps with fewer than 100 dies are removed, leaving 172,946. A stratified random sample of 10,000 maps forms the test set (Center 248, Donut 32, Edge-Loc 300, Edge-Ring 560, Loc 208, Random 50, Scratch 69, Near-full 9, None 8,524); the remaining 162,946 form the training set. Because None covers 85% of the maps, macro-averaged F1 is the main metric; micro-F1 equals accuracy. Fig. 1 shows one test map per class.

Handcrafted features (MFE). The 59 features of [9]: 13 defect densities over fixed wafer regions, 40 Radon-transform features (20 cubic-interpolated row means and 20 row standard

deviations), and 6 geometric properties of the most salient defect region (area, perimeter, major and minor axis length, eccentricity, solidity). We use the authors’ public feature-extraction code [24] unchanged.

Base classifiers. *MFE+FNN*: standardized features, two hidden layers of 128 tanh units, softmax output. *CNN*: the map is binarized (defective die = 1), resized to 64×64 , scaled to $[-1, 1]$ and replicated to three channels; the VGG16 convolutional stack [12], initialized from ImageNet [25], is followed by global average pooling and a softmax layer. Both are trained with Adam [26] (learning rate 10^{-4} , batch 32) and early stopping on validation loss (patience 20), as in [9].

Meta-classifiers. Let $p_1, p_2 \in \mathbb{R}^9$ be the two class-probability vectors and $x = [p_1; p_2]$. The proposed MLR meta-classifier is a ridge regression [27] of the one-hot label on x (no intercept, $\alpha = 0.1$); the predicted class is the arg-max of the nine outputs, so each class receives its own weight for each base classifier. Baselines: Stacking-FNN (one hidden layer of 10 units), Stacking-DT (a default scikit-learn decision tree [28]), and MultiNN, a CNN whose pooled 512-dimensional features are concatenated with the 59 handcrafted features before the output layer [29].

IV. EXPERIMENTAL PROTOCOL

Splits. The training set is split into A (the first 130,356 maps, 80%) and B (the remaining 32,590). Base classifiers are fitted on A ; B is split once, stratified, into halves B_{fit} and B_{val} (16,295 each). Base classifiers early-stop on B_{fit} , so their outputs on B are not in-sample.

Meta-training. The original protocol fits the meta-classifier on the base classifiers’ outputs for their own training data (our “A-mode”). We additionally fit it on B (“B-mode”), where the base outputs behave like test-time outputs. For model selection, stackers are scored by two-fold cross-fitting over B ($B_{\text{fit}} \leftrightarrow B_{\text{val}}$, predictions pooled over all of B); final B-mode stackers are fitted on 80% of B (the remaining 20% serve the FNN’s early stopping).

Selection discipline. Every choice—hyperparameters, which extension to keep, the final pipeline—is made on B . The test set is used only to report results. Reported variation is the standard deviation over runs, where runs differ in the MFE+FNN seed (3 or 5 seeds; MLR is deterministic given its inputs) or in the stacker seed.

Statistics. Comparisons report the paired t -test across runs, as in [9], and a paired bootstrap [30] over wafers (2,000 resamples) of the macro-F1 difference, giving a 95% interval that reflects the limited number of rare-class wafers. **Inclusion rule:** an extension is kept if it improves the B score by more than the baseline’s run-to-run standard deviation; extensions that fail are reported.

Implementation. TensorFlow [31] (2.10 and 2.17; equivalence checked by re-predicting a trained CNN within 1.2×10^{-4}), scikit-learn, scikit-image [32] and XGBoost [33]; single NVIDIA GPUs. A CNN trains in about two hours.

TABLE I

REPRODUCTION AT $N = 162,946$: TEST MACRO- AND MICRO-F1, OURS VS. KANG AND KANG [9] (TABLE 7, 10 REPLICATIONS). STACKING-DT AND STACKING-MLR USE THE ORIGINAL PROTOCOL (META-CLASSIFIER FITTED ON IN-SAMPLE OUTPUTS ON A).

Model	macro-F1		micro-F1		runs
	ours	reported	ours	reported	
MFE+FNN	0.8572 ± 0.0062	0.8599 ± 0.0117	0.9735 ± 0.0003	0.9741 ± 0.0010	3 seeds
CNN	0.8777	0.8679 ± 0.0126	0.9790	0.9775 ± 0.0014	1 run
MultiNN	0.8880	0.8455 ± 0.0170	0.9794	0.9728 ± 0.0015	1 run
Stacking-DT	0.8782 ± 0.0065	0.8789 ± 0.0094	0.9752 ± 0.0005	0.9757 ± 0.0011	3 seeds
Stacking-FNN [†]	0.9041 ± 0.0028	0.8991 ± 0.0096	0.9811 ± 0.0002	0.9808 ± 0.0010	5 seeds
Stacking-MLR	0.8967 ± 0.0060	0.8949 ± 0.0121	0.9805 ± 0.0004	0.9801 ± 0.0010	3 seeds

[†]Hidden layer of 64 units, fitted on B ; with the original $18 \rightarrow 10 \rightarrow 9$ architecture on in-sample outputs the stack reached 0.8402 ± 0.0393 (class-weighted) or 0.6298 ± 0.0904 (unweighted).

V. REPRODUCTION

A. Full training set

Table I compares our reproduction with Table 7 of [9]. All six models land within or above one reported standard deviation. The stack improves on both of its base classifiers, as claimed.

Three implementation details that the reference code handles differently were decisive. First, *class weights hurt*: the reference code trains the base classifiers with balanced class weights, but the paper mentions none; removing them raised the MFE+FNN from 0.8147 to the reported level and also helped the CNN. Second, ImageNet initialization and flip/rotation augmentation were needed for the CNN to reach the reported score (a from-scratch, class-weighted CNN reached 0.8496). Third, mixed precision and larger batches cost about four macro-F1 points on this VGG network without batch normalization, so all CNNs use 32-bit arithmetic and batch 32.

B. The meta-training pitfall

Fitting the meta-classifier on in-sample outputs, as in the reference implementation, produced a stack that never predicted Near-full ($F1 = 0$) although both base classifiers recognized it, with macro-F1 0.6298 ± 0.0904 . In-sample, the base classifiers are almost always right and nearly certain, so the meta-classifier learns to follow the arg-max and never learns when to distrust a base classifier; rare classes, seen a handful of times, are swamped. Class weights mask the problem but leave it unstable across seeds (0.8402 ± 0.0393). Fitting the meta-classifier on B , where outputs are as uncertain as at test time, removes it (Table I). This is the classic recommendation of stacking [10], [11], but it is easy to miss, and the MLR stack of the paper—which is linear and has few parameters—is far less affected than the FNN or tree meta-classifiers.

C. Training-size sweep

Table II and Fig. 2 repeat the sweep of [9] with the original protocol: for each replicate, N training maps are sampled at random, 80% are used for fitting and 20% for early stopping, and the meta-classifiers are fitted on the fitting part. At $N = 5,000$ and $50,000$ every model exceeds the reported value; our

TABLE II

TRAINING-SIZE SWEEP, TEST MACRO-F1 (OURS / REPORTED IN [9]). ORIGINAL PROTOCOL: IN-SAMPLE META-TRAINING ON THE FITTING PART. LAST ROW: FULL-DATA REPRODUCTION (TABLE I).

N	reps	MFE+FNN	CNN	Stack-DT	Stack-FNN	Stack-MLR
500	10	0.462 / 0.556	0.490 / 0.494	0.268 / 0.518	0.435 / 0.508	0.514 / 0.587
5,000	10	0.729 / 0.698	0.788 / 0.695	0.739 / 0.722	0.798 / 0.735	0.804 / 0.760
50,000	5	0.837 / 0.831	0.883 / 0.840	0.853 / 0.842	0.886 / 0.875	0.876 / 0.869
162,946	1–5	0.857 / 0.860	0.878 / 0.868	0.878 / 0.879	0.904 / 0.899	0.897 / 0.895

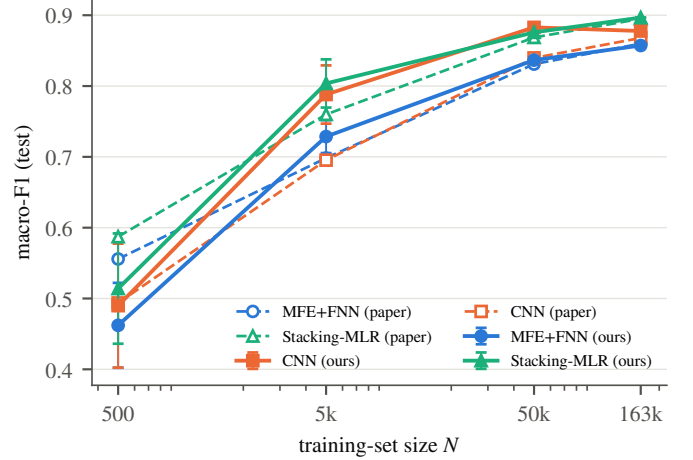


Fig. 2. Test macro-F1 against training-set size. Solid lines with filled markers: ours (mean $\pm$ std over replicates); dashed lines with hollow markers: reported in [9].

CNN gains most (+0.09 at $N = 5,000$), which we attribute to augmentation and the absence of class weights. At $N = 500$ our CNN matches the reported value, while the handcrafted-feature model and the stackers fall below it. Both sides show large variation there: with 400 fitting maps, several rare classes are often absent altogether (Donut and Near-full in our first replicate).

The sweep also shows the pitfall at work. With in-sample meta-training, Stacking-DT collapses at $N = 500$ (0.268) because the tree learns thresholds on near-one-hot probabilities that do not transfer to the test set. At $N = 50,000$ the MLR stack (0.876) falls *below* our CNN (0.883): the CNN looks near-perfect in-sample, so the stack cannot learn when to override it. The paper’s claim that the stack beats both base classifiers therefore holds at $N = 500$, $5,000$ and $162,946$ in our runs, but not at $50,000$ under the original protocol.

VI. EXTENSIONS

All extensions start from the corrected pipeline: MFE+FNN (5 seeds) and the CNN, both without class weights, with the meta-classifier fitted on B . Table III lists every configuration; Table IV gives the paired comparisons.

A. Test-time augmentation (TTA)

Wafer defect patterns keep their class under rotation and reflection, so the CNN’s prediction is averaged over the eight symmetries of the square (four rotations, with and without a

TABLE III

EXTENSIONS. B : TWO-FOLD CROSS-FITTED MACRO-F1 ON B (SELECTION); TEST: MACRO-F1 ON THE 10,000 TEST MAPS. CNN $\times$ 5: AVERAGE OF FIVE CNN SEEDS, EACH WITH TEST-TIME AUGMENTATION. ROWS WITHOUT $\pm$: A SINGLE DETERMINISTIC RUN.

Configuration	B (MLR)	test (MLR)	test (FNN)
Stack-2: MFE-FNN + CNN	0.9030 $\pm$ 0.0034	0.9005 $\pm$ 0.0076	0.9004 $\pm$ 0.0048
+ TTA on the CNN	0.9074 $\pm$ 0.0045	0.9051 $\pm$ 0.0069	0.9067 $\pm$ 0.0062
+ XGBoost as third learner	0.9105 $\pm$ 0.0030	0.9149 $\pm$ 0.0025	0.9070 $\pm$ 0.0037
Control: XGBoost replaces MFE-FNN	0.9123	0.9160	0.9130 $\pm$ 0.0022
Control: 2nd MFE-FNN seed as third learner	0.9094 $\pm$ 0.0040	0.9059 $\pm$ 0.0051	0.9029 $\pm$ 0.0057
MFE-FNN + CNN $\times$ 5	0.9165 $\pm$ 0.0057	0.9075 $\pm$ 0.0054	0.9095 $\pm$ 0.0029
MFE-FNN + CNN $\times$ 5 + XGBoost	0.9182 $\pm$ 0.0039	0.9125 $\pm$ 0.0015	0.9065 $\pm$ 0.0034
CNN $\times$ 5 + XGBoost (final)	0.9223	0.9134	0.9092 $\pm$ 0.0023

TABLE IV

PAIRED COMPARISONS (DIFFERENCE IN MACRO-F1, t -TEST ACROSS RUNS, 95% WAFER-BOOTSTRAP INTERVAL).

Comparison	set	Δ	p	95% CI
TTA (MLR)	B	+0.0044	0.011	[-0.0004, +0.0095]
TTA (FNN)	B	+0.0061	0.009	[+0.0011, +0.0121]
XGBoost as 3rd learner (MLR)	B	+0.0031	0.199	[-0.0037, +0.0108]
XGBoost as 3rd learner (FNN)	B	+0.0050	0.019	[-0.0007, +0.0117]
XGBoost vs. 2nd MFE-FNN seed (MLR)	B	+0.0011	0.713	[-0.0056, +0.0087]
XGBoost vs. 2nd MFE-FNN seed (FNN)	B	+0.0023	0.354	[-0.0031, +0.0089]
CNN $\times$ 5 vs. CNN $\times$ 1 (final stack)	B	+0.0100	—	[+0.0030, +0.0178]
adding MFE-FNN + XGB to CNN $\times$ 5 ‡	B_{val}	-0.0062	—	[-0.0175, +0.0036]
final vs. reproduced Stacking-MLR	test	+0.0168	—	[+0.0004, +0.0399]
final vs. Stack-2 + TTA (MLR)	test	+0.0083	—	[-0.0079, +0.0292]

‡ Stack MFE-FNN + CNN $\times$ 5 + XGB vs. the CNN $\times$ 5 ensemble alone, on B_{val} .

flip), without retraining. TTA passes the inclusion rule for both meta-classifiers on B (Table IV) and raises the test macro-F1 of both stacks.

B. A gradient-boosted third learner

We added XGBoost [33] on the same 59 features as a third base classifier, fitted on A with the number of trees chosen by early stopping on B_{fit} . A 12-configuration grid (Appendix) was scored by the B score of the resulting stack; the selected model (depth 6, learning rate 0.05, balanced class weights) is also the best on its own (B_{val} macro-F1 0.8704 against 0.8525 for MFE+FNN). Two controls separate the possible explanations: replacing the MFE+FNN with XGBoost (“is XGBoost simply better?”) and filling the third slot with a second MFE+FNN seed (“does any third model help?”). The third learner passes the inclusion rule only for the FNN meta-classifier, and it never beats the second-seed control significantly on B , so we find no evidence that a *different kind* of learner helps. Replacing the MFE+FNN with XGBoost gives the best B score among single-CNN stacks.

C. CNN seed ensembles

Four further CNNs with the same recipe and different seeds reach the same validation loss (0.053–0.054) yet differ by ± 0.012 in B_{val} macro-F1 (Appendix, Table VII): the rare classes swing between seeds. Averaging the five TTA-CNNs [20] gives the largest single improvement in this study (+0.010 on B for the XGBoost stack, interval excluding zero).

TABLE V

REJECT OPTION FOR THE FINAL PIPELINE ON THE TEST SET. THRESHOLDS CHOSEN ON B .

target	coverage	accuracy	macro-F1
100%	1.000	0.9821	0.9134
98%	0.983	0.9902	0.9537
95%	0.952	0.9972	0.9861
90%	0.902	0.9996	0.9976
85%	0.857	0.9998	0.9990
80%	0.796	0.9999	0.9993

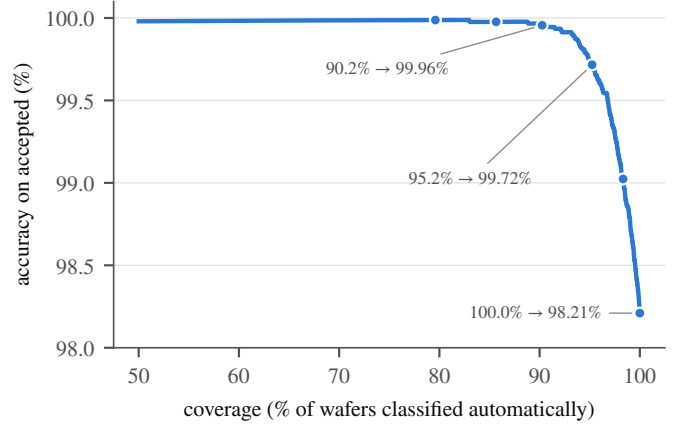


Fig. 3. Risk-coverage curve of the final pipeline on the test set; markers: operating points with thresholds chosen on B (labels: coverage $\rightarrow$ accuracy).

D. Final pipeline

On B , the best configuration is the five-seed CNN ensemble with TTA plus XGBoost, combined by MLR. It reaches a test macro-F1 of 0.9134 and an accuracy of 98.21%, a gain of +0.0168 over our reproduced Stacking-MLR with a bootstrap interval excluding zero. On the test set, the same stack with a single CNN scored slightly higher (0.9160); the difference is well inside the bootstrap interval (-0.0025 , $[-0.0143, +0.0091]$), and the ensemble was chosen on B . Per-class F1 is given in the Appendix (Table VI).

VII. SELECTIVE PREDICTION

In a fab, an uncertain wafer can be sent to an engineer. We convert the MLR scores to probabilities with a temperature-scaled softmax (temperature $T = 0.133$ fitted by log-loss on the cross-fitted B predictions) and accept a wafer if its maximum probability exceeds a threshold. Thresholds are chosen on B for target coverages and then applied to the test set unchanged (Table V, Fig. 3). Classifying 95.2% of wafers automatically yields 99.72% accuracy and a macro-F1 of 0.9861 on them; the deferred wafers are mostly Loc (44% of that class), Scratch (38%), Edge-Loc (35%) and Donut (25%), while only 2% of None wafers are deferred, which keeps the manual workload small.

VIII. WHAT THE STACK LEARNS

Fig. 4 shows the diagonal MLR weights—the weight each base classifier’s vote for a class receives in that class’s score—

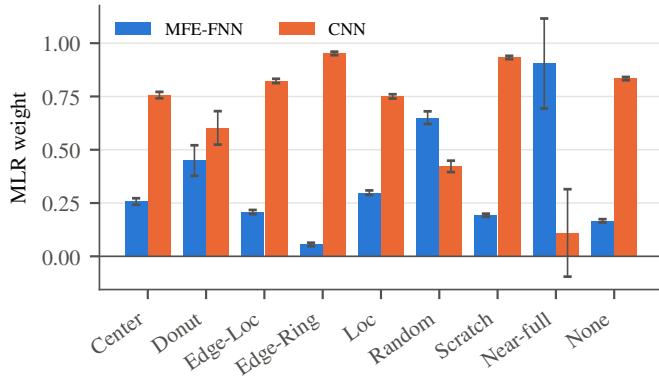


Fig. 4. MLR weight of each base classifier’s own-class vote (MFE-FNN + CNN with TTA; mean $\pm$ std over five MFE+FNN seeds).

for the two-learner stack with TTA. As in Fig. 5 of [9], the handcrafted-feature model carries Random and Near-full, which are defined by global density, while the CNN carries the spatial patterns such as Edge-Ring and Scratch. In the three-learner stack, XGBoost takes over most of the Near-full weight from the MFE+FNN. TreeSHAP [34], [35] on the XGBoost model ranks geometric features (eccentricity, solidity) highest for Scratch and the area of the defect region highest for Near-full; Grad-CAM [36] maps of the CNN concentrate on the defect regions for correctly classified wafers (Appendix, Fig. 5).

IX. NEGATIVE RESULTS AND LIMITATIONS

Negative results. (i) Once the CNN is a five-seed ensemble, the original MFE+FNN no longer helps: the stack with it scores below the CNN ensemble alone on B_{val} (Table IV); handcrafted features still contribute a little through XGBoost. (ii) A third learner of a different kind is not better than a second seed of the existing one. (iii) Bagging XGBoost over five subsampled seeds improved it on its own (B_{val} 0.8785) but lowered every stack it entered on B by 0.0014–0.0018, so it was dropped. (iv) Temperature scaling did not improve calibration of the MLR stack: the clipped and renormalized MLR outputs already have a test ECE of 0.0041, against 0.0070 after scaling; it is still needed to turn MLR scores into probabilities for the reject rule. (v) Class weights, mixed precision and a reduce-on-plateau learning-rate schedule did not help. (vi) Under in-sample meta-training, Stacking-MLR does not beat the CNN at $N = 50,000$.

Limitations. We use one fixed train/test split, whereas [9] averages ten random splits; our variation reflects seeds of the learners, and the wafer bootstrap covers sampling of the test set. Near-full has only nine test wafers, so one wafer moves the macro-F1 by about 0.01. The test set was consulted twice—after the first round of extensions and for the final pipeline—although every selection was made on B . The training-size sweep uses five replicates at $N = 50,000$ instead of ten, MultiNN is a single run, and the preliminary model-selection experiments of [9] (their Tables 5–6) were not repeated.

X. CONCLUSION

The stacking ensemble of Kang and Kang reproduces: at full data our Stacking-MLR reaches 0.8967 against the reported 0.8949, and the stack beats both base classifiers at three of four training sizes. Two practical lessons emerge. The meta-classifier must be trained on outputs for data the base classifiers did not fit, or it learns to trust over-confident in-sample predictions. And the value of the handcrafted-feature branch depends on how strong the CNN is: against a single CNN it helps, against a five-seed ensemble with test-time augmentation it no longer does. Our best pipeline reaches a macro-F1 of 0.9134 and, with a reject option, classifies 95.2% of wafers at 99.72% accuracy.

CODE AVAILABILITY

Code, logs and result files are available at https://github.com/Kartikeya2046/ML_wafer_detecting_using_stack_ensemble. Every table in this paper is generated from the saved outputs by a script in that repository.

REFERENCES

- [1] M.-J. Wu, J.-S. R. Jang, and J.-L. Chen, “Wafer map failure pattern recognition and similarity ranking for large-scale data sets,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 28, pp. 1–12, 2015.
- [2] M. Piao, C. H. Jin, J. Y. Lee, and J. Byun, “Decision tree ensemble-based wafer map failure pattern recognition based on radon transform-based features,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 31, pp. 250–257, 2018.
- [3] M. Saqlain, B. Jargalsaikhan, and J. Y. Lee, “A voting ensemble classifier for wafer map defect patterns identification in semiconductor manufacturing,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 32, pp. 171–182, 2019.
- [4] T. Nakazawa and D. V. Kulkarni, “Wafer map defect pattern classification and image retrieval using convolutional neural network,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 31, pp. 309–314, 2018.
- [5] K. Kyeong and H. Kim, “Classification of mixed-type defect patterns in wafer bin maps using convolutional neural networks,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 31, pp. 395–402, 2018.
- [6] N. Yu, Q. Xu, and H. Wang, “Wafer defect pattern recognition and analysis based on convolutional neural network,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 32, pp. 566–573, 2019.
- [7] M. Saqlain, Q. Abbas, and J. Y. Lee, “A deep convolutional neural network for wafer defect identification on an imbalanced dataset in semiconductor manufacturing processes,” *IEEE Transactions on Semiconductor Manufacturing*, vol. 33, pp. 436–444, 2020.
- [8] S. Kang, “Rotation-invariant wafer map pattern classification with convolutional neural networks,” *IEEE Access*, vol. 8, pp. 170 650–170 658, 2020.
- [9] H. Kang and S. Kang, “A stacking ensemble classifier with handcrafted and convolutional features for wafer map pattern classification,” *Computers in Industry*, vol. 129, p. 103450, 2021.
- [10] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [11] K. M. Ting and I. H. Witten, “Issues in stacked generalization,” *Journal of Artificial Intelligence Research*, vol. 10, pp. 271–289, 1999.
- [12] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [13] MIR Lab, “LSWMD (wm-811k) wafer map dataset,” <http://mirlab.org/dataSet/public/>, 2018.
- [14] J. Pineau, P. Vincent-Lamarre, K. Sinha *et al.*, “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.

- [15] J. Yu and X. Lu, "Wafer map defect detection and recognition using joint local and nonlocal linear discriminant analysis," *IEEE Transactions on Semiconductor Manufacturing*, vol. 29, pp. 33–43, 2016.
- [16] J. Shim, S. Kang, and S. Cho, "Active learning of convolutional neural network for cost-effective wafer map pattern classification," *IEEE Transactions on Semiconductor Manufacturing*, vol. 33, pp. 258–266, 2020.
- [17] T.-H. Tsai and Y.-C. Lee, "A light-weight neural network for wafer map classification based on data augmentation," *IEEE Transactions on Semiconductor Manufacturing*, vol. 33, pp. 663–672, 2020.
- [18] C. H. Jin, H.-J. Kim, Y. Piao, M. Li, and M. Piao, "Wafer map defect pattern classification based on convolutional neural network features and error-correcting output codes," *Journal of Intelligent Manufacturing*, vol. 31, pp. 1861–1875, 2020.
- [19] S. Džeroski and B. Ženko, "Is combining classifiers with stacking better than selecting the best one?" *Machine Learning*, vol. 54, pp. 255–273, 2004.
- [20] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems 30*, 2017.
- [21] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.
- [22] M. P. Naeini, G. F. Cooper, and M. Hauskrecht, "Obtaining well calibrated probabilities using Bayesian binning," in *Proc. 29th AAAI Conference on Artificial Intelligence*, 2015, pp. 2901–2907.
- [23] Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems 30*, 2017.
- [24] DMkelllog, "WMPC_Stacking_TF2: Wafer map pattern classification using stacking ensemble," https://github.com/DMkelllog/WMPC_Stacking_TF2, 2021.
- [25] O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei, "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015.
- [26] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.
- [27] A. E. Hoerl and R. W. Kennard, "Ridge regression: Biased estimation for nonorthogonal problems," *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [28] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [29] DMkelllog, "wafermap_MultiNN: Wafer map defect pattern classification with multi-input neural network," https://github.com/DMkelllog/wafermap_MultiNN, 2021.
- [30] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman & Hall, 1993.
- [31] M. Abadi *et al.*, "TensorFlow: A system for large-scale machine learning," in *Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2016, pp. 265–283.
- [32] S. van der Walt, J. L. Schönberger, J. Nunez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu, "scikit-image: Image processing in Python," *PeerJ*, vol. 2, p. e453, 2014.
- [33] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [34] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems 30*, 2017.
- [35] S. M. Lundberg, G. Erion, H. Chen *et al.*, "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, pp. 56–67, 2020.
- [36] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

APPENDIX

TABLE VI

PER-CLASS TEST F1: REPORTED STACKING-MLR [9] (TABLE 8), OUR REPRODUCTION, AND OUR FINAL PIPELINE.

Class	n	reported	reproduction	final
Center	248	0.9424	0.9553	0.9579
Donut	32	0.8969	0.8915	0.8889
Edge-Loc	300	0.8492	0.8434	0.8636
Edge-Ring	560	0.9823	0.9866	0.9920
Loc	208	0.7950	0.7742	0.7837
Random	50	0.8979	0.8821	0.9000
Scratch	69	0.7928	0.8254	0.8421
Near-full	9	0.9053	0.9191	1.0000
None	8524	0.9918	0.9922	0.9926
Macro	10,000	0.8948	0.8967	0.9134

TABLE VII

CNN SEEDS (SAME RECIPE). MACRO-F1 ON B_{val} AND TEST.

seed	B_{val}	B_{val} (TTA)	test (TTA)
0	0.8723	0.8797	0.8781
1	0.8850	0.9053	0.8879
2	0.8974	0.9143	0.8948
3	0.8858	0.8987	0.8883
4	0.8802	0.8933	0.9129
mean $\pm$ std		0.8983 ± 0.0116	0.8924 ± 0.0116
ensemble (5)		0.9221	0.9126

TABLE VIII

XGBOOST GRID, ORDERED BY THE B SCORE OF THE THREE-LEARNER MLR STACK (SELECTION CRITERION).

depth	lr	class weights	trees	B_{val} alone	stack (B)
6	0.05	balanced	1260	0.8704	0.9105 ± 0.0030
6	0.1	balanced	589	0.8667	0.9097 ± 0.0027
8	0.1	balanced	314	0.8645	0.9095 ± 0.0024
4	0.05	none	992	0.8665	0.9093 ± 0.0028
4	0.1	balanced	1564	0.8695	0.9091 ± 0.0021
4	0.1	none	473	0.8607	0.9088 ± 0.0036
8	0.05	balanced	688	0.8674	0.9088 ± 0.0019
6	0.1	none	209	0.8596	0.9084 ± 0.0034
6	0.05	none	442	0.8602	0.9081 ± 0.0035
4	0.05	balanced	3187	0.8675	0.9074 ± 0.0024
8	0.1	none	125	0.8536	0.9073 ± 0.0016
8	0.05	none	259	0.8532	0.9072 ± 0.0013

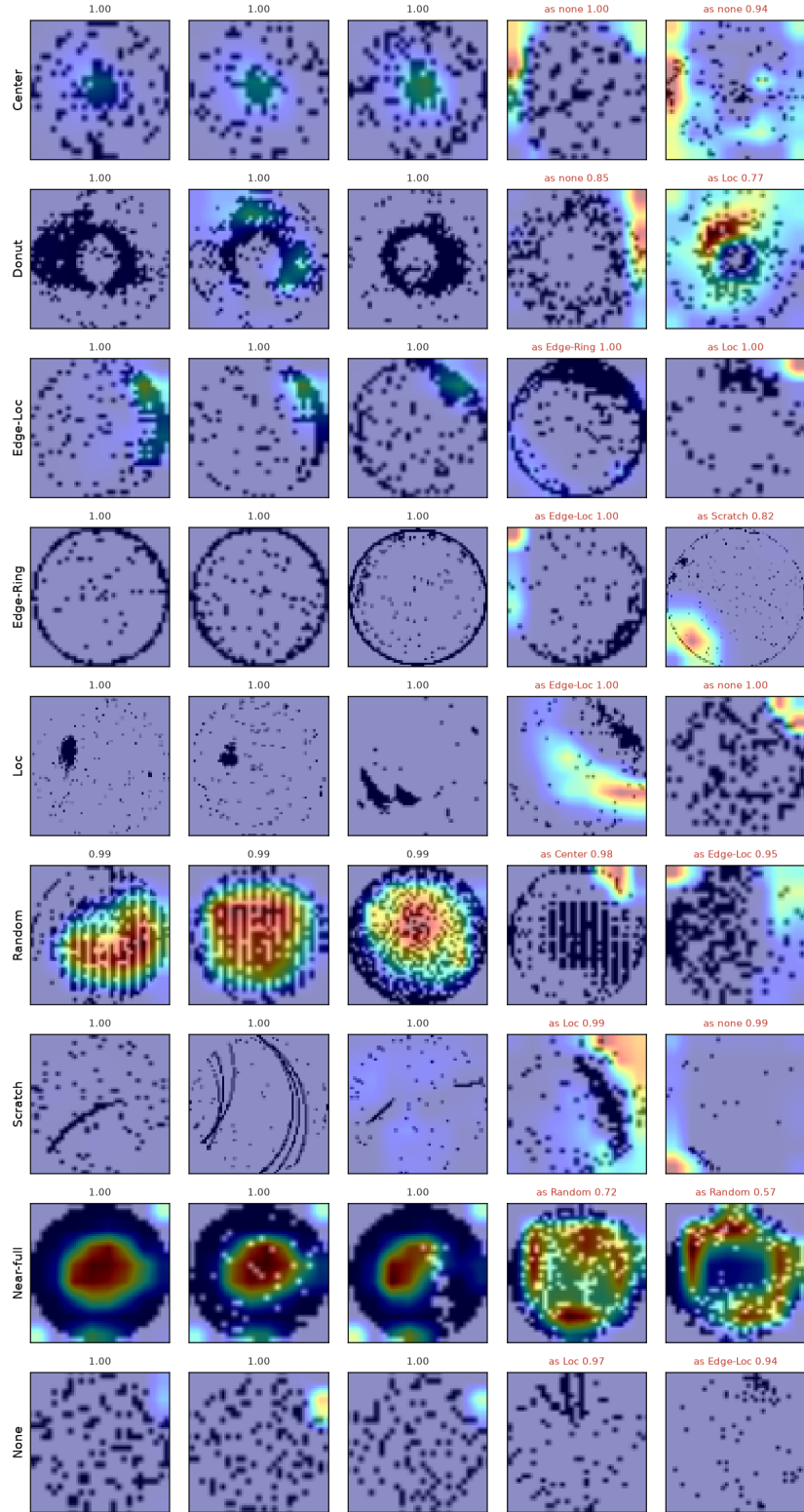


Fig. 5. Grad-CAM [36] on test wafers (layer block4_conv3, 8×8): three correctly classified maps per class (left) and up to two misclassified maps (right, red titles). Confident correct maps of some classes are nearly blank because the softmax is saturated, a known limitation of gradient-based maps.